

MICROSERVICES COURSE

Spring Boot & Spring Cloud — Professional Edition

Session 17 — Observability Deep Dive

Distributed Tracing · Metrics · Structured Logging

Micrometer + Zipkin + Prometheus + Grafana

1. Session Overview

In previous sessions, we built a microservices platform with:

- Spring Cloud Gateway
- Service Discovery
- Centralized Configuration
- Circuit Breakers
- Retries and Fallbacks
- Bulkheads
- OpenFeign
- Kafka
- Distributed Sagas
- Caching
- Testing
- Docker

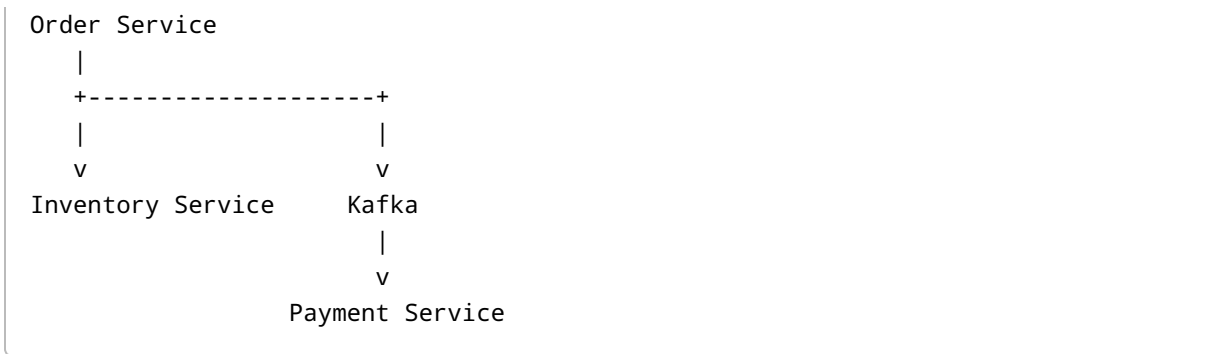
Our platform is now distributed.

This is a major achievement.

But it also creates a new problem.

A request may now travel through multiple services:

```
Client
|
v
API Gateway
|
v
```



If something fails, where do we look?

Which service was slow?

Which service generated the error?

How do we find every log related to one specific request?

This is the purpose of **Observability**.

2. Learning Objectives

By the end of this session, you will be able to:

The Observability Problem

- Distinguish between Monitoring and Observability
- Explain why separate service logs are difficult to investigate
- Understand the Three Pillars of Observability
- Explain how Traces, Metrics, and Logs work together

Distributed Tracing

- Define Trace and Span
- Understand parent/child span relationships
- Understand trace context propagation
- Configure Micrometer Tracing
- Send traces to Zipkin
- Investigate a distributed request in the Zipkin UI
- Understand tracing across synchronous and asynchronous communication

Metrics

- Explain Micrometer's role in Spring Boot

- Distinguish between Counter, Gauge, and Timer
- Create custom metrics
- Expose Prometheus metrics
- Configure Prometheus scraping
- Write basic PromQL queries
- Visualize metrics using Grafana

Structured Logging

- Understand the limitations of plain-text logs
- Configure JSON logs
- Include trace IDs in logs
- Correlate logs with Zipkin traces

3. The Observability Problem

3.1 The 9-Second Order

Imagine that a customer says:

"I placed an order. It took 9 seconds. I am not even sure whether it worked."

You start investigating.

You open the Order Service logs.

```
2026-08-31 10:15:02 INFO OrderService - Creating order
2026-08-31 10:15:11 INFO OrderService - Order confirmed
```

The order took approximately 9 seconds.

But why?

You open the Gateway logs.

You see separate timestamps.

Then you open the Inventory Service logs.

Then the Payment Service logs.

Then Kafka consumer logs.

Now you start manually comparing timestamps.

Gateway:	10:15:02.100
Order:	10:15:02.130
Inventory:	10:15:03.500
Payment:	10:15:08.700

Was the Inventory Service slow?

Was there a retry?

Did the Circuit Breaker activate?

Was Kafka delayed?

Was a database query slow?

Without observability, you are guessing.

4. Monitoring vs Observability

Monitoring

Monitoring answers questions that we already know to ask.

Examples:

Is CPU above 80%?

Is Order Service UP?

Is JVM memory high?

Is the error rate increasing?

Monitoring normally uses:

- Dashboards

- Alerts
- Health checks
- Metrics

Monitoring tells us:

Something is wrong.

Observability

Observability helps us answer questions that we did not know in advance.

Example:

Why did THIS specific order take 9 seconds?

Which service caused the delay?

What happened during this customer's request?

Observability tells us:

Here is where and why something happened.

5. The Three Pillars of Observability

Modern observability is usually based on three complementary data types.

Pillar	Main Question
Metrics	How is the system behaving over time?
Traces	Where did this specific request spend time?
Logs	What exactly happened?

5.1 Metrics

Metrics are numerical measurements over time.

Examples:

```
CPU usage
JVM memory
Request count
Error count
Request duration
```

Metrics answer:

How is the system behaving?

Example:

```
P99 request latency increased from 300ms to 9 seconds.
```

5.2 Traces

Traces follow one request across multiple services.

Example:

```
Gateway
|
v
Order Service
|
+--> Inventory Service
|
+--> Kafka
      |
      v
    Payment Service
```

Traces answer:

Where did the time go?

5.3 Logs

Logs provide detailed information about what happened.

Example:

```
InventoryService - Product PROD-001 is out of stock
```

Logs answer:

Why did this operation fail?

5.4 The Complete Picture

Imagine:

```
P99 latency suddenly increases.
```

Metrics tell us:

Something is wrong.

Then we open a trace.

```
Order Service -> Inventory Service = 8.5 seconds
```

Traces tell us:

The Inventory Service call is slow.

Then we search logs.

```
Database connection pool exhausted
```

Logs tell us:

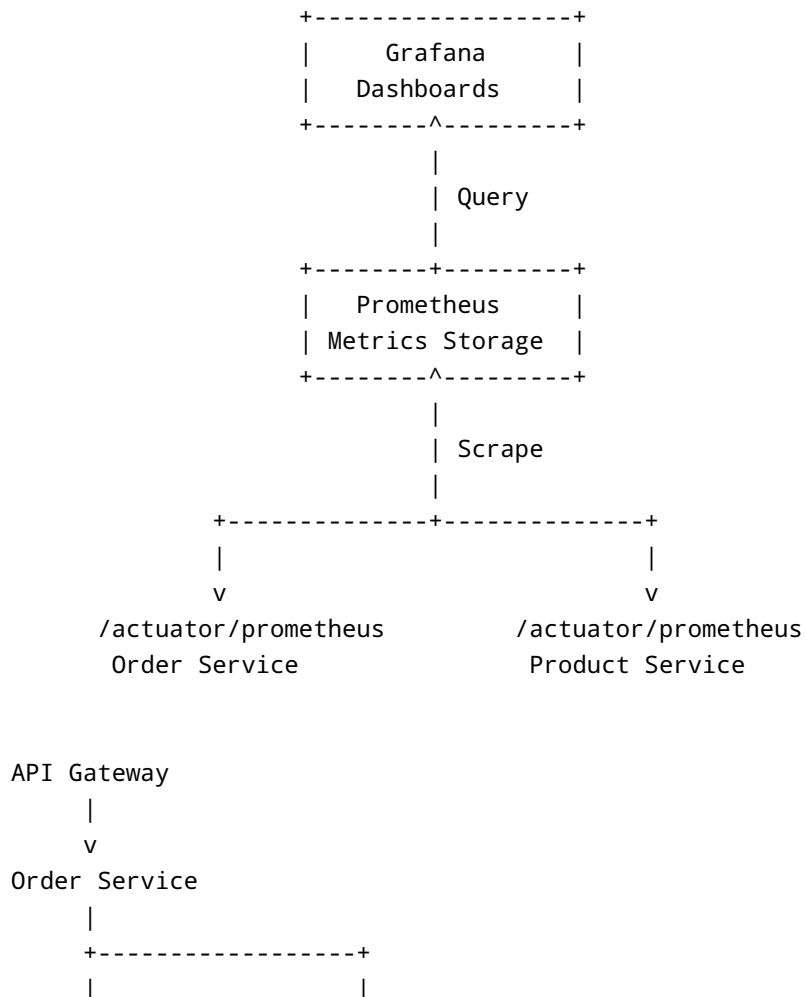
This is why.

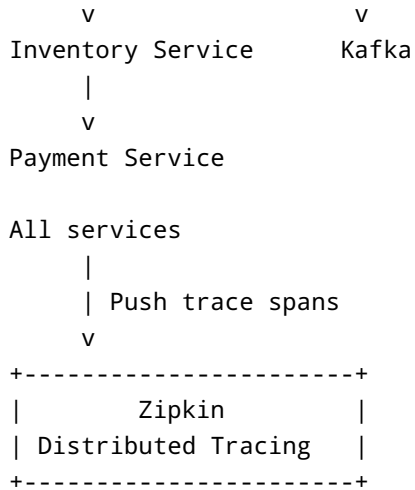
The complete flow is:

```
METRICS
Something is wrong
  |
  v
TRACES
Where is the problem?
  |
  v
LOGS
Why did it happen?
```

6. Observability Architecture

Our platform will use:





Every service also produces:

Structured JSON Logs
+
Trace ID
+
Span ID

PART 1 — DISTRIBUTED TRACING

7. What Is a Trace?

A **Trace** represents the complete journey of one request.

Example:

Client sends:

POST /api/orders

The request travels through:

Gateway
|
v

```
Order Service
|
v
Inventory Service
|
v
Kafka
|
v
Payment Service
```

All these operations can belong to one trace.

Example:

```
Trace ID:

7a3f9c2b1e8d4f6a
```

Every participating service can know that the request belongs to:

```
7a3f9c2b1e8d4f6a
```

8. What Is a Span?

A **Span** represents one unit of work inside a trace.

Example:

```
Trace: 7a3f9c2b1e8d4f6a

Gateway request
|
+-- Order Service createOrder()
    |
    +-- Feign call to Inventory
        |
        +-- Kafka event publishing
            |
            +-- Payment Kafka consumer
```

Each operation can create a span.

Example:

```
Trace ID: 7a3f9c2b1e8d4f6a
```

All spans share the same trace ID.

But each span has its own span ID.

```
Trace ID: 7a3f9c2b1e8d4f6a  
Span ID: 1111
```

```
Trace ID: 7a3f9c2b1e8d4f6a  
Span ID: 2222
```

```
Trace ID: 7a3f9c2b1e8d4f6a  
Span ID: 3333
```

9. Parent and Child Spans

Spans form a hierarchy.

Example:

```
Gateway Span  
|  
+-- Order Service Span  
    |  
    +-- Inventory Feign Span  
        |  
        +-- Kafka Producer Span  
            |  
            +-- Payment Consumer Span
```

This relationship allows tracing tools to show the complete request path.

10. Trace Context Propagation

A trace is only useful if every service receives the same trace context.

The context normally contains information such as:

```
Trace ID
Current Span ID
Parent information
```

The context must travel with the request.

10.1 HTTP Propagation

Example:

```
Gateway
|
| HTTP headers
v
Order Service
|
| HTTP headers
v
Inventory Service
```

Modern tracing commonly uses standard trace propagation headers such as:

```
traceparent
```

The next service extracts the context and continues the trace.

10.2 Kafka Propagation

For asynchronous communication:

```
Order Service
|
| Kafka Message
```

```
      | + Trace Context Headers
      v
Kafka
  |
  v
Payment Service
```

The Kafka consumer receives the trace context.

This allows the distributed trace to continue across asynchronous boundaries.

11. Sampling

Tracing every request can create a large amount of data.

Therefore, tracing systems may use **sampling**.

Example:

```
1.0 = 100%
```

Trace every request.

```
0.1 = 10%
```

Trace approximately 10% of requests.

For this course:

```
management:
  tracing:
    sampling:
      probability: 1.0
```

This means:

```
Trace 100% of requests.
```

This is appropriate for:

- Local development
- Learning
- Testing

In high-traffic production environments, the sampling strategy should be selected according to traffic, storage, cost, and operational requirements.

12. Zipkin

Zipkin is a distributed tracing system.

Our services will:

```
Create spans
  |
  v
Send spans
  |
  v
Zipkin
  |
  v
Zipkin UI
```

Zipkin will help us answer:

Which service was slow?

How long did every operation take?

Where did the request fail?

13. Start Zipkin

Our course infrastructure already includes Zipkin.

Check Docker:

```
docker compose ps
```

You should see Zipkin running.

Example:

```
zipkin    running    0.0.0.0:9411->9411/tcp
```

Open:

```
http://localhost:9411
```

At the beginning of the session, Zipkin may be empty.

That is expected.

14. Add Tracing Dependencies

Add these dependencies to every service that participates in the traced request.

For example:

- API Gateway
- Order Service
- Product Service
- Inventory Service
- Payment Service

pom.xml

```
<dependency>
  <groupId>io.micrometer</groupId>
  <artifactId>micrometer-tracing-bridge-brave</artifactId>
</dependency>

<dependency>
  <groupId>io.zipkin.reporter2</groupId>
```

```
<artifactId>zipkin-reporter-brave</artifactId>
</dependency>
```

We also need Actuator.

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
```

Actuator should already exist in our services.

15. Configure Micrometer Tracing

Add the following configuration.

```
management:

  tracing:
    sampling:
      probability: 1.0

  zipkin:
    tracing:
      endpoint: http://localhost:9411/api/v2/spans
```

This configuration means:

```
Sample every request
  |
  v
Create tracing data
  |
  v
Send spans to Zipkin
```

16. Service Name

Each service should have a meaningful application name.

Example:

```
spring:
  application:
    name: order-service
```

This is important because observability tools need to identify the service.

Zipkin can show:

```
gateway-service
order-service
inventory-service
payment-service
```

Meaningful service names make traces understandable.

17. Verify a Distributed Trace

Restart your services.

Then generate a request.

Example:

```
curl -X POST http://localhost:8080/api/orders
-H "Content-Type: application/json"
-d '{
  "productId": "PROD-001",
  "quantity": 1,
  "amount": 100.00
}'
```

Open Zipkin:

```
http://localhost:9411
```

Select:

Service Name -> order-service

Then:

Run Query

You should see a trace.

Open the trace.

You should see spans similar to:

```
Gateway
|
+-- Order Service
    |
    +-- Inventory Service
```

18. Reading the Zipkin Waterfall

A trace normally shows a waterfall-style visualization.

Example:

```
Gateway
[=====]

Order Service
  [=====]

Inventory Service
    [====]

Kafka
      [===]
```

The horizontal length represents time.

If one span is much longer:

```
Inventory Service
[=====]
```

Then that operation may be responsible for the delay.

19. Tracing Across Kafka

Our platform also uses asynchronous communication.

Example:

```
Order Service
  |
  v
Kafka Event
  |
  v
Payment Service
```

Tracing must cross this asynchronous boundary.

With Spring Kafka and Micrometer tracing support, Kafka instrumentation can propagate tracing context through message headers.

The conceptual flow is:

```
HTTP Request
  |
  | Trace ID = ABC123
  v
Order Service
  |
  | Kafka headers
  | Trace ID = ABC123
  v
Kafka
  |
  v
Payment Consumer
  |
  | Continues Trace ABC123
```

```
v
Payment Service
```

This is important because the consumer may run:

- On another thread
- In another service
- Later in time

But the business operation can still be connected to the same distributed trace.

CHECKPOINT 1

At this point, verify:

- Zipkin container is running
- Zipkin UI opens
- Services start successfully
- A request creates a trace
- Multiple services appear in the trace

Do not continue until this works.

PART 2 — METRICS

20. What Is Micrometer?

Micrometer is a metrics abstraction layer.

A useful analogy is:

```
SLF4J
|
+--> Logback
+--> Log4j2
```

Similarly:

```
Micrometer
|
```

```
+--> Prometheus  
+--> Datadog  
+--> Cloud Monitoring  
+--> Other monitoring systems
```

Your application records metrics through Micrometer.

Micrometer can then expose those metrics to a specific monitoring system.

21. Built-In Metrics

Spring Boot Actuator already provides many metrics.

Examples:

```
JVM memory  
JVM threads  
CPU  
Garbage collection  
HTTP requests  
Database pools
```

The metrics may already be collected by the application.

Today we will expose them to Prometheus.

22. Metric Types

22.1 Counter

A Counter answers:

How many times did something happen?

Examples:

```
Orders created  
Payments failed  
Messages consumed
```

A counter only moves in one direction.

0
1
2
3
4

22.2 Gauge

A Gauge answers:

What is the current value right now?

Examples:

Active users
Queue size
Available connections

A gauge can move in both directions.

10
20
15
7
25

22.3 Timer

A Timer answers:

How long did something take?

Examples:

```
Order creation duration
Database query duration
HTTP request duration
```

Timers can provide information about:

```
Count
Total time
Maximum time
Distribution
```

23. Add Prometheus Support

Add this dependency:

```
<dependency>
  <groupId>io.micrometer</groupId>
  <artifactId>micrometer-registry-prometheus</artifactId>
</dependency>
```

Restart the service.

24. Expose the Prometheus Endpoint

Configure Actuator.

```
management:
  endpoints:
    web:
      exposure:
        include: health,info,prometheus
```

For the course platform, keep any existing actuator endpoints that were already configured in previous sessions.

The important addition is:

```
prometheus
```

25. Verify Prometheus Metrics

Open:

```
http://localhost:8082/actuator/prometheus
```

The port depends on your Order Service configuration.

You should see text output similar to:

```
jvm_memory_used_bytes
```

```
http_server_requests_seconds_count
```

```
http_server_requests_seconds_sum
```

This is the format that Prometheus understands.

26. Create a Custom Counter

We will count successfully created orders.

OrderMetricsService.java

```
package com.example.orderservice.metrics;

import io.micrometer.core.instrument.Counter;
import io.micrometer.core.instrument.MeterRegistry;
import org.springframework.stereotype.Component;

@Component
public class OrderMetricsService {

    private final Counter ordersCreatedCounter;
```

```

public OrderMetricsService(MeterRegistry meterRegistry) {

    this.ordersCreatedCounter =
        Counter.builder("orders.created")
            .description("Number of successfully created orders")
            .register(meterRegistry);
}

public void incrementOrdersCreated() {
    ordersCreatedCounter.increment();
}
}

```

Then inject it into your order processing logic.

Example:

```

@Service
public class OrderService {

    private final OrderMetricsService orderMetricsService;

    public OrderService(OrderMetricsService orderMetricsService) {
        this.orderMetricsService = orderMetricsService;
    }

    public OrderResponse createOrder(OrderRequest request) {

        // Create order

        OrderResponse response = new OrderResponse();

        // Business logic

        orderMetricsService.incrementOrdersCreated();

        return response;
    }
}

```

After creating orders, check:

```
/actuator/prometheus
```

You should find:

```
orders_created_total
```

Prometheus metric names can be transformed from the Java metric name into a Prometheus-compatible name.

27. Measure Method Duration with @Timed

Add:

```
import io.micrometer.core.annotation.Timed;
```

Then:

```
@Timed(
    value = "order.create.duration",
    description = "Time required to create an order"
)
@PostMapping
public CompletableFuture<ResponseEntity<OrderResponse>>
createOrder(@RequestBody OrderRequest request) {

    return orderService
        .createOrderAsync(request)
        .thenApply(ResponseEntity::ok);
}
```

`@Timed` measures how long the annotated operation takes.

28. Important Note About @Timed

Depending on the exact Spring Boot and Micrometer configuration used by the project, annotation-based timing may require the `TimedAspect` to be registered.

Example:

```
@Configuration
public class MetricsConfiguration {

    @Bean
    public TimedAspect timedAspect(MeterRegistry meterRegistry) {
        return new TimedAspect(meterRegistry);
    }
}
```

Add this only when required by your project's configuration.

Always verify the actual metric output.

29. Metric Tags

Metrics can have tags.

Example:

```
orders.created
```

With tags:

```
orders.created
service=order-service
status=success
```

Tags help us filter and group metrics.

For example:

```
orders.created
status=success
```

and:

```
orders.created
status=failed
```

This is powerful.

But there is an important rule.

30. High Cardinality

Do not create metrics using highly unique values as tags.

Bad:

```
Counter.builder("orders.created")  
    .tag("orderId", orderId)
```

If there are millions of orders:

```
orderId=1  
orderId=2  
orderId=3  
orderId=4  
...
```

Prometheus must store a huge number of unique time series.

This is called **high cardinality**.

Avoid using:

```
Order ID  
Customer ID  
Email  
Trace ID  
Random UUID
```

as metric labels.

Good tags usually have a limited number of possible values.

Example:

```
status=success
status=failed
```

PART 3 — PROMETHEUS

31. What Is Prometheus?

Prometheus is a metrics monitoring system and time-series database.

Prometheus periodically requests metrics from our services.

```
Prometheus
  |
  | GET /actuator/prometheus
  v
Order Service
```

This is called:

Scraping

32. Pull Model

Prometheus usually pulls metrics.

```
Prometheus
  |
  | Pull
  v
Application
```

This is different from Zipkin.

```
Application
  |
  | Push
```

v
Zipkin

Remember:

Prometheus = Pull Metrics
Zipkin = Receive Trace Spans

33. Create prometheus.yml

Create:

prometheus/prometheus.yml

Example:

```
global:
  scrape_interval: 15s

scrape_configs:
  - job_name: 'order-service'

    metrics_path: '/actuator/prometheus'

    static_configs:
      - targets:
        - 'order-service:8082'

  - job_name: 'product-service'

    metrics_path: '/actuator/prometheus'

    static_configs:
      - targets:
        - 'product-service:8081'
```

34. Docker Networking

This is extremely important.

When Prometheus runs inside Docker:

```
localhost
```

means:

```
The Prometheus container itself
```

It does NOT mean:

```
Your Windows, Linux, or macOS machine
```

Therefore:

```
localhost:8082
```

may be incorrect inside the Prometheus container.

If both services are running in the same Docker network:

```
order-service:8082
```

works because:

```
order-service
```

is the Docker service name or resolvable container hostname.

35. Docker Compose Configuration

Add Prometheus.

```
prometheus:
  image: prom/prometheus:latest
  container_name: prometheus

  volumes:
    - ./prometheus/prometheus.yml:/etc/prometheus/prometheus.yml

  ports:
    - "9090:9090"

  networks:
    - platform-net
```

Add Grafana.

```
grafana:
  image: grafana/grafana:latest
  container_name: grafana

  ports:
    - "3000:3000"

  networks:
    - platform-net
```

Make sure:

```
prometheus
grafana
order-service
product-service
```

are connected to the required network.

Example:

```
networks:
  platform-net:
    driver: bridge
```

36. Start Prometheus and Grafana

Run:

```
docker compose up -d prometheus grafana
```

Verify:

```
docker compose ps
```

Expected:

```
prometheus    running
grafana       running
```

37. Verify Prometheus

Open:

```
http://localhost:9090
```

Then navigate to:

```
Status
|
v
Targets
```

You should see your services.

Example:

```
order-service

State: UP
```

If the target is:

DOWN

Prometheus cannot access the metrics endpoint.

Do not continue until the target is:

UP

38. Basic PromQL

Prometheus uses a query language called:

PromQL

Open:

`http://localhost:9090`

Then use the query area.

38.1 Request Count

Try:

`http_server_requests_seconds_count`

38.2 Custom Order Metric

Try:

`orders_created_total`

38.3 Requests Per Second

Example:

```
rate(http_server_requests_seconds_count[1m])
```

This calculates the approximate rate of change over the previous minute.

38.4 Average Request Duration

Example:

```
rate(http_server_requests_seconds_sum[1m])  
/  
rate(http_server_requests_seconds_count[1m])
```

This demonstrates a very important Prometheus principle:

```
Raw counters  
  |  
  v  
PromQL calculations  
  |  
  v  
Operational information
```

CHECKPOINT 2

At this point, verify:

- `/actuator/prometheus` works
 - Prometheus is running
 - Prometheus targets are UP
 - PromQL returns metrics
 - Custom order metrics appear
-

PART 4 — GRAFANA

39. What Is Grafana?

Grafana visualizes data.

The architecture is:

```
Application
  |
  v
Prometheus
  |
  v
Grafana
```

Grafana does not normally collect metrics from your Spring Boot service.

Instead:

```
Grafana
  |
  | Query
  v
Prometheus
```

40. Open Grafana

Open:

```
http://localhost:3000
```

For the standard local Docker image, the default initial login is commonly:

```
Username: admin
Password: admin
```

If you changed credentials using Docker environment variables, use the credentials configured for your environment.

Grafana may ask you to change the password.

41. Add Prometheus as a Data Source

In Grafana:

```
Connections
|
v
Data Sources
|
v
Add Data Source
|
v
Prometheus
```

If Grafana and Prometheus are inside the same Docker network:

`http://prometheus:9090`

Use this URL.

Do not use:

`http://localhost:9090`

because inside the Grafana container:

`localhost`

means the Grafana container.

Save and test the data source.

You should see a successful connection.

42. Grafana Explore

Before importing a dashboard, use:

Explore

Select:

Prometheus

Try:

orders_created_total

Run the query.

You should see data.

This proves:

```
Spring Boot
  |
  v
Prometheus
  |
  v
Grafana
```

is working.

43. Create Your First Dashboard

Create a dashboard manually.

Select:

Dashboards

|

```
v
New
|
v
New Dashboard
|
v
Add Visualization
```

Select:

Prometheus

44. Suggested Dashboard Panels

Panel 1 — Order Creation Rate

Example query:

```
rate(orders_created_total[1m])
```

Panel 2 — HTTP Request Rate

Example:

```
sum(rate(http_server_requests_seconds_count[1m]))
```

Panel 3 — Request Duration

Use an appropriate HTTP request duration metric from your service.

Panel 4 — JVM Memory

Example:

```
jvm_memory_used_bytes
```

Panel 5 — Custom Order Metric

```
orders_created_total
```

Now your dashboard might look conceptually like:

```
+-----+-----+
| Request Rate   | Request Duration |
+-----+-----+
| Order Count    | JVM Memory       |
+-----+-----+
```

45. Imported Dashboards

After learning to create a basic dashboard manually, you may import a community dashboard.

However:

Never assume that an imported dashboard will work perfectly with every Spring Boot application.

Why?

Because:

- Metric names can differ
- Labels can differ
- Micrometer versions can differ
- Application configuration can differ

Therefore:

```
First understand your metrics.
Then build a simple dashboard.
Then optionally import dashboards.
```

PART 5 — STRUCTURED LOGGING

46. The Problem with Plain-Text Logs

Example:

```
2026-08-31 10:15:02 INFO OrderService - Order created
```

This is acceptable for:

```
One application  
One log file
```

But imagine:

```
API Gateway  
Order Service  
Product Service  
Inventory Service  
Payment Service  
Notification Service  
Kafka consumers
```

Now imagine searching for:

```
Everything related to one request.
```

Searching plain text becomes difficult.

47. Structured Logging

Structured logging represents log information as fields.

Example:

```
{  
  "@timestamp": "2026-08-31T10:15:02.123Z",  
  "level": "INFO",
```

```
"service": "order-service",  
"traceId": "7a3f9c2b1e8d4f6a",  
"spanId": "91ab3c",  
"message": "Order created"  
}
```

This allows systems to search by:

```
service  
level  
traceId  
spanId  
message  
timestamp
```

48. Add JSON Logging Dependency

Add:

```
<dependency>  
  <groupId>net.logstash.logback</groupId>  
  <artifactId>logstash-logback-encoder</artifactId>  
</dependency>
```

For a production course project, dependency versions should normally be managed consistently through your project's dependency management strategy.

49. Create logback-spring.xml

Create:

```
src/main/resources/logback-spring.xml
```

Example:

```
<configuration>  
  
  <appender name="JSON"
```

```
        class="ch.qos.logback.core.ConsoleAppender">

        <encoder
            class="net.logstash.logback.encoder.LogstashEncoder">

            <includeMdcKeyName>traceId</includeMdcKeyName>

            <includeMdcKeyName>spanId</includeMdcKeyName>

        </encoder>

    </appender>

    <root level="INFO">

        <appender-ref ref="JSON"/>

    </root>

</configuration>
```

Restart the application.

50. Trace and Log Correlation

Micrometer tracing integrates tracing context with the logging context.

The logging context is commonly represented using:

MDC

MDC means:

Mapped Diagnostic Context

Conceptually:

```
Request
|
+-- traceId
|
+-- spanId
|
```

v
Application Log

Now logs can contain:

traceId
spanId

Example:

```
{  
  "message": "Creating order",  
  "traceId": "ABC123",  
  "spanId": "111"  
}
```

Another service:

```
{  
  "message": "Inventory checked",  
  "traceId": "ABC123",  
  "spanId": "222"  
}
```

The span IDs differ.

But the trace ID connects the complete request.

51. Verify Trace Correlation

Send a request through the Gateway.

Example:

```
POST /api/orders
```

Find the trace ID.

Then observe logs.

Example:

Gateway

traceId=ABC123

Order Service

traceId=ABC123

Inventory Service

traceId=ABC123

Now:

Zipkin Trace ID

|

v

ABC123

|

+--> Gateway Logs

|

+--> Order Logs

|

+--> Inventory Logs

This is called:

Log Correlation

52. Structured Logging vs Centralized Logging

Structured logging:

JSON format

Centralized logging:

```
Collect logs from many services
  |
  v
Store centrally
  |
  v
Search centrally
```

Examples of centralized logging systems include ELK-based architectures.

This session focuses on:

```
Structured JSON Logs
+
Trace Correlation
```

We are not building a complete centralized logging platform in this session.

PART 6 — END-TO-END OBSERVABILITY INVESTIGATION

53. The Incident

A customer reports:

```
My order took 9 seconds.
```

We will investigate.

Step 1 — Check Metrics

Open Grafana.

Suppose we see:

```
P99 latency increased
```

Metrics tell us:

Something is wrong.

Step 2 — Find a Slow Trace

Open Zipkin.

Find a trace with approximately:

9 seconds

The trace shows:

Gateway	9.0 seconds
Order Service	8.9 seconds
Inventory Call	8.5 seconds

Traces tell us:

The Inventory call is responsible for most of the delay.

Step 3 — Copy the Trace ID

Example:

7a3f9c2b1e8d4f6a

Step 4 — Search Logs

Search for:

traceId=7a3f9c2b1e8d4f6a

Example:

```
{
  "message": "Database connection timeout",
  "traceId": "7a3f9c2b1e8d4f6a"
}
```

Logs tell us:

The database connection was slow or unavailable.

54. The Observability Workflow

Always think:

```
METRICS
  |
  | Something is wrong
  v
TRACES
  |
  | Where is it?
  v
LOGS
  |
  | Why did it happen?
  v
ROOT CAUSE
```

This is the core operational workflow.

PART 7 — COMPLETE DOCKER COMPOSE EXAMPLE

The following is a simplified observability infrastructure example.

```
services:

  zipkin:

    image: openzipkin/zipkin:latest

    ports:
```

```

      - "9411:9411"

networks:
  - platform-net

prometheus:

  image: prom/prometheus:latest

  ports:
    - "9090:9090"

  volumes:
    - ./prometheus/prometheus.yml:/etc/prometheus/prometheus.yml

  networks:
    - platform-net

grafana:

  image: grafana/grafana:latest

  ports:
    - "3000:3000"

  networks:
    - platform-net

networks:

  platform-net:

    driver: bridge

```

Your actual course project may already contain:

- PostgreSQL
- Config Server
- Eureka
- Kafka
- Redis
- Other infrastructure

Do not replace your existing Docker Compose configuration.

Add the observability services to the existing platform configuration.

PART 8 — TROUBLESHOOTING

55. Zipkin Shows No Traces

Check:

```
docker compose ps
```

Verify Zipkin:

```
http://localhost:9411
```

Check tracing dependencies.

Check:

```
management:
  tracing:
    sampling:
      probability: 1.0
```

Check the Zipkin endpoint.

Verify that a real request reaches the application.

56. Prometheus Target Is DOWN

Open:

```
http://localhost:9090/targets
```

Check the target.

Verify:

```
http://SERVICE:PORT/actuator/prometheus
```

Check:

- Service is running
- Port is correct
- Prometheus can reach the service
- Docker networking is correct
- Actuator exposure includes `prometheus`

57. Grafana Cannot Connect to Prometheus

Remember:

```
Grafana container localhost
```

means:

```
Grafana container itself
```

If both containers are on the same Docker network, use:

```
http://prometheus:9090
```

Verify:

```
docker compose ps
```

Both containers should be running.

58. Custom Metric Does Not Appear

Check:

- Did the code execute?
- Did the counter increment?
- Did you restart the application?

- Are you checking `/actuator/prometheus`?
- Is Prometheus scraping the service?

Generate several requests.

Then check again.

59. Logs Do Not Contain Trace IDs

Check:

- Micrometer tracing dependency
- Active request context
- JSON logging configuration
- MDC fields in the encoder
- Application restart

Remember:

A trace ID exists during a traced operation.

A log created outside a traced context may not have the same correlation information.

60. Zipkin Shows Only One Service

Check:

- Does the request actually call the second service?
 - Are tracing dependencies present in both services?
 - Is propagation working?
 - Is the second service configured correctly?
 - Is the second service sending spans to Zipkin?
-

PART 9 — STUDENT LAB

61. Lab — Build Observability for the Platform

Task 1 — Enable Distributed Tracing

Configure tracing for:

- API Gateway
- Order Service
- Inventory Service

Requirements:

- Micrometer tracing
 - Zipkin reporting
 - 100% development sampling
-

Task 2 — Generate a Distributed Trace

Send a request:

```
Gateway
  |
  v
Order Service
  |
  v
Inventory Service
```

Verify that all services appear in Zipkin.

Task 3 — Create a Custom Metric

Create:

```
orders.created
```

Requirements:

- Use Micrometer
 - Increment after successful order creation
 - Verify the metric through Actuator
-

Task 4 — Configure Prometheus

Configure Prometheus to scrape:

- Order Service
- Product Service

Verify:

```
Status -> Targets -> UP
```

Task 5 — Write PromQL

Run at least:

```
orders_created_total
```

and:

```
rate(http_server_requests_seconds_count[1m])
```

Task 6 — Create a Grafana Dashboard

Create at least four panels:

1. Request rate
 2. Request duration
 3. Order count
 4. JVM memory
-

Task 7 — Configure Structured JSON Logging

Configure:

```
JSON
traceId
spanId
```

Generate a request.

Verify that multiple services log the same trace ID.

62. Lab Acceptance Criteria

Your implementation is complete when you can demonstrate:

1. Send an order request
|
v
2. Find the request in Zipkin
|
v
3. Identify all participating services
|
v
4. See metrics in Prometheus
|
v
5. See dashboards in Grafana
|
v
6. Find logs using the trace ID

PART 10 — FINAL CHECKLIST

Before completing the session:

Distributed Tracing

- ☐ Zipkin is running
- ☐ Zipkin UI opens
- ☐ Services send traces
- ☐ A distributed trace contains multiple services
- ☐ Trace context propagates through service calls

Metrics

- ☐ Prometheus dependency exists
- ☐ `/actuator/prometheus` works
- ☐ Custom metric exists
- ☐ Prometheus target is UP
- ☐ PromQL query works

Grafana

- ☐ Grafana is running
- ☐ Prometheus is configured as a data source
- ☐ A metric is visible in Explore
- ☐ A dashboard was created

Logging

- ☐ JSON logs are generated
- ☐ Logs contain trace ID
- ☐ Logs contain span ID
- ☐ The same trace ID can be used to investigate multiple services

63. Key Concepts Summary

Monitoring

Is something wrong?

Observability

What happened, where, and why?

Metrics

How is the system behaving?

Traces

Where did this request go?

Logs

What exactly happened?

Micrometer

Application metrics and tracing instrumentation.

Zipkin

Distributed tracing visualization and investigation.

Prometheus

Metrics scraping and time-series storage.

Grafana

Metrics visualization and dashboards.

Trace ID

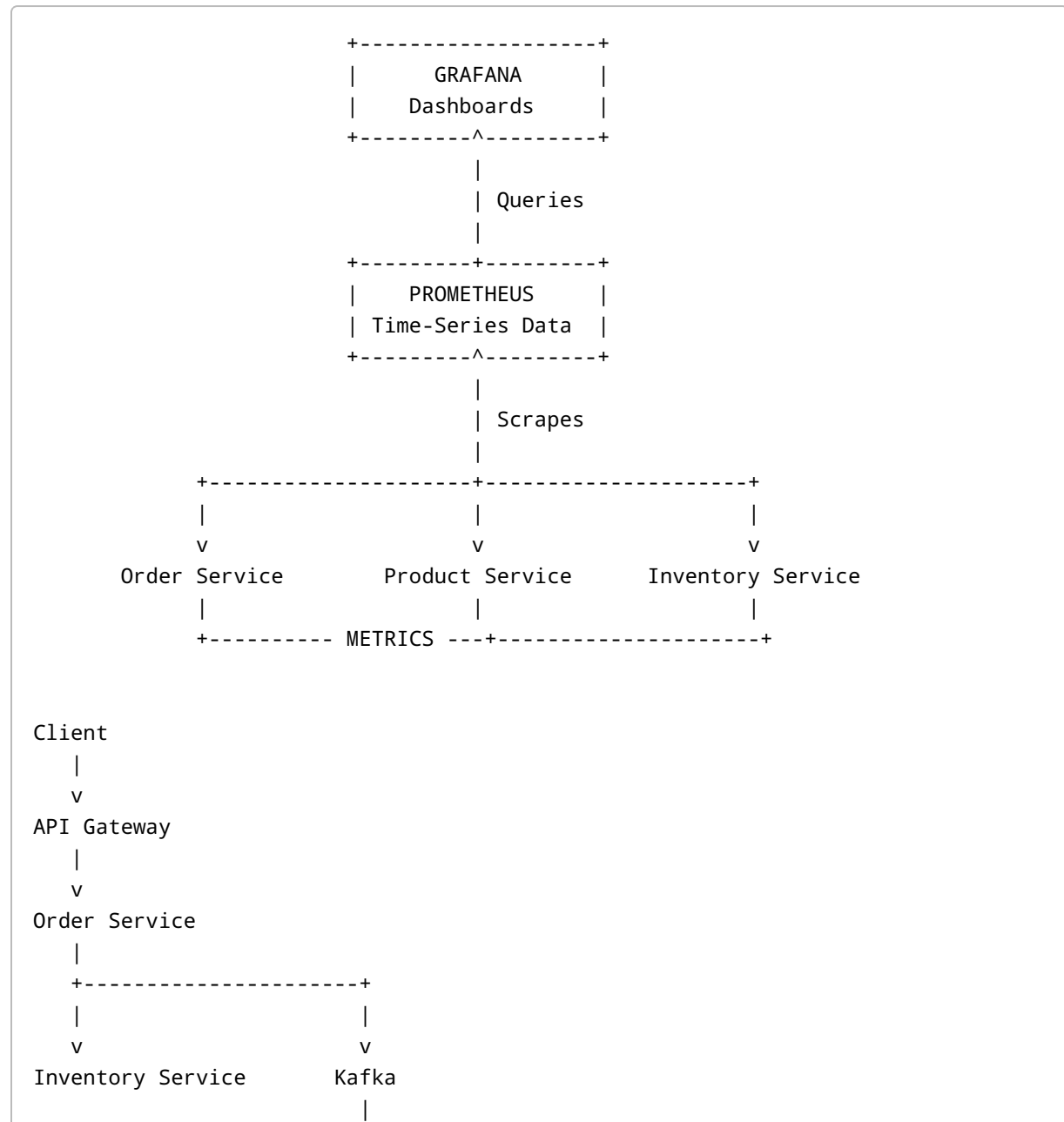
Identifies the complete distributed request.

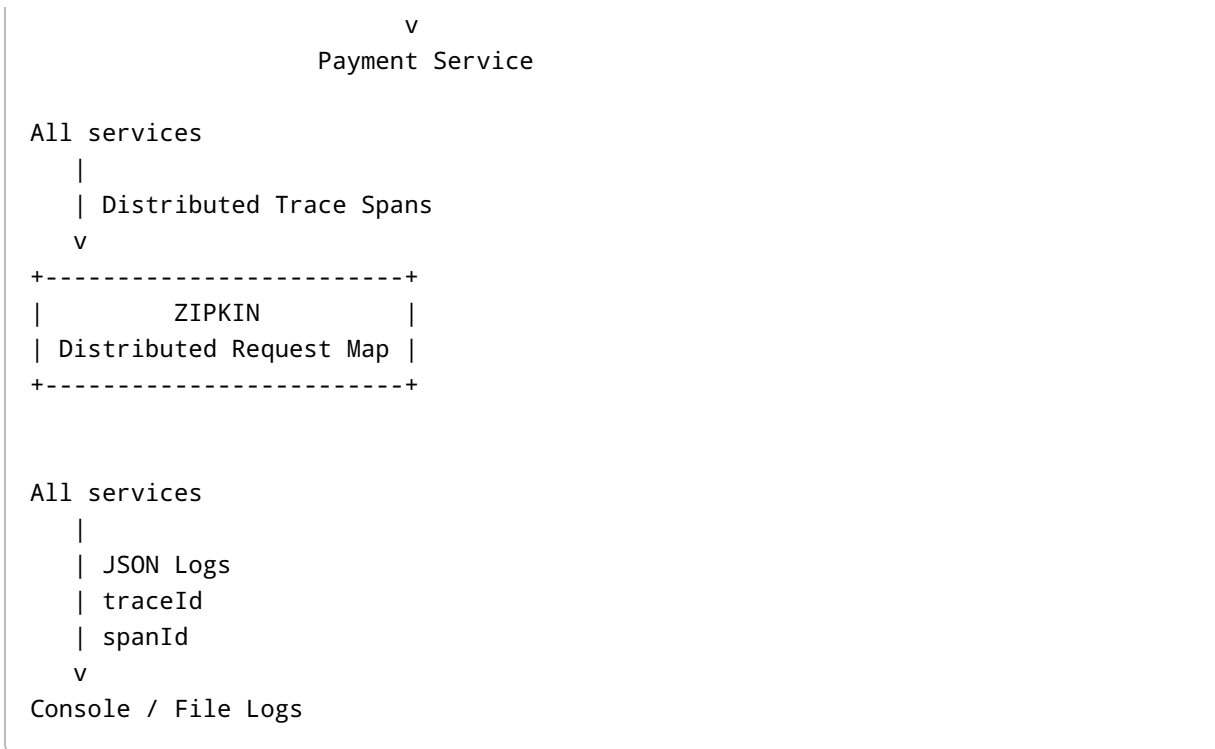
Span ID

Identifies one operation inside the trace.

64. Final Architecture

After this session, our platform observability architecture is:





65. Final Takeaway

Microservices give us:

Independent services
Independent deployment
Independent scaling
Fault isolation

But they also create:

Distributed complexity

A production-quality microservices platform must not only:

Run

It must also be:

Observable

The complete operational flow is:

```
Metric Alert
  |
  v
Find Slow Trace
  |
  v
Identify Slow Service
  |
  v
Search Correlated Logs
  |
  v
Find Root Cause
```

This is the foundation of observability in a modern Spring Boot microservices platform.

End of Session 17

Next Session

Advanced Patterns — CQRS & Event Sourcing

We will move from:

Observing distributed systems

to:

Designing systems around separate command and query models

and explore:

- CQRS
- Command Model
- Query Model
- Event Sourcing

- Event Store
- Axon concepts
- Read projections
- Eventual consistency
- When CQRS is useful
- When CQRS should be avoided